Solving the N-Queens Problem with SAT: A Comparison of AMO/ALO Encodings, SAT Solvers, Constraint Programming and Integer Programming

Hai[a]

[a]Faculty of Information Technology, VNU University of Engineering and Technology, 144 Xuan Thuy, Cau Giay, Hanoi, Vietnam

ARTICLE INFO

Keywords:
N-Queens
SAT encoding
at-most-one (AMO)
PySAT
constraint programming
integer linear programming

ABSTRACT

The N-Queens problem asks for a placement of N queens on an $N \times N$ board such that no two queens attack each other. We solve it with Boolean satisfiability (SAT) and compare SAT with two other exact paradigms: constraint programming (CP) and integer linear programming (ILP). For SAT, we implement five at-most-one (AMO) encodings (pairwise, binary, sequential counter, commander, product), test four further encodings from the PySAT library, and run eight CDCL solvers, including CaDiCaL, Glucose, Kissat and CryptoMiniSat. We then compare the best SAT configuration with Google OR-Tools CP-SAT, IBM CPLEX CP Optimizer, CPLEX MIP and Gurobi. All encodings are verified by counting all solutions for $N = 2, \ldots, 9$, which match the known values (OEIS A000170). Under a 30-second, single-thread budget we find that: (i) the *commander* encoding is the most robust, solving $N = 512$ in 11.4 s with CaDiCaL; (ii) a small number of clauses does not guarantee fast solving: sequential counter is compact but times out from $N = 256$; (iii) the best SAT configuration reaches $N = 1000$ in 27.7 s, further than CP-SAT ($N \leq 256$); and (iv) CPLEX CP is the fastest method where its community licence allows it to run ($N \leq 128$), while Gurobi and CPLEX MIP are limited by their licences to $N \leq 44$ and $N \leq 31$.

1. Introduction

Motivation. The N-Queens problem is a classical benchmark in artificial intelligence and operations research [2, 10]. A solution exists for every $N \geq 4$ and can even be built directly in $O(N)$ time [2], so the problem is not hard in theory. However, it is a good *controlled testbed*: the same simple constraints can be modelled in very different ways, and we can measure how each modelling choice affects performance.

Three paradigms.

- **SAT** writes the board as a propositional formula in conjunctive normal form (CNF) and solves it with a conflict-driven clause-learning (CDCL) solver [4, 11]. Performance depends strongly on how the "at most one queen per line" constraint is encoded [5, 7, 9, 21].
- **Constraint programming (CP)** keeps the constraint as a global `AllDifferent` constraint and uses a dedicated propagator [13].
- **Integer linear programming (ILP)** writes the constraints as linear (in)equalities over 0/1 variables and solves them by branch-and-bound/cut.

Research questions. We structure the study around three questions:

RQ1 How does the choice of AMO encoding affect the CNF size and the solving time?

RQ2 How do different SAT solvers perform on the same encoding?

RQ3 How does the best SAT configuration compare with CP (CP-SAT, CPLEX CP) and ILP (Gurobi, CPLEX MIP)?

ORCID(s):

Contributions.

1. We implement five AMO encodings and use four PySAT encodings, and verify all of them by exhaustive solution counting for $N = 2, \dots, 9$.
2. We derive closed-form clause counts for the pairwise and sequential-counter encodings and confirm them against the implementation.
3. We benchmark the encodings on eight CDCL solvers on boards up to 1024×1024.
4. We compare SAT with CP-SAT, CPLEX CP, Gurobi and CPLEX MIP, and state clearly up to which N each commercial solver is limited by its licence.

Organisation. Section 2 gives the background. Section 3 describes our models. Section 4 describes the experimental setup, Section 5 reports the results, Section 6 discusses them, and Section 7 concludes.

2. Background

2.1. The N-Queens problem

We index the cells by (i, j) with $0 \le i, j < N$. A set Q of N cells is a solution if, for every two distinct cells $(i, j), (i', j') \in Q$:

$$i \neq i', \qquad j \neq j', \qquad i - j \neq i' - j', \qquad i + j \neq i' + j'. \tag{1}$$

So every **row** and every **column** contains *exactly one* queen, and every **diagonal** contains *at most one* queen. (There are $2N - 1$ diagonals in each direction but only N queens, so "exactly one" on diagonals would be unsatisfiable.) The number of solutions for $N = 2, \dots, 9$ is $0, 0, 2, 10, 4, 40, 92, 352$; we use these values to validate our encodings (Section 5.1). Fig. 1 shows one solution for $N = 8$ and the four lines that pass through one cell.

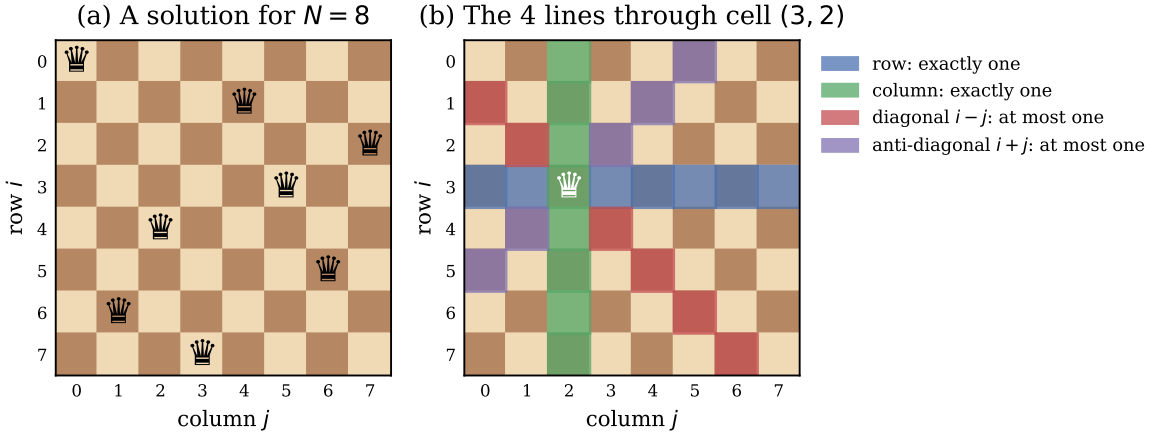


Figure 1: (a) One solution for $N = 8$ (the queen of row i is in column $0, 4, 7, 5, 2, 6, 1, 3$). (b) Each cell lies on one row, one column, one diagonal and one anti-diagonal. Rows and columns must contain exactly one queen; diagonals at most one.

Lines. We call every row, column, diagonal ($i - j = d$) and anti-diagonal ($i + j = s$) a *line*. Each direction has $2N - 1$ diagonals, of which the two corner diagonals have length 1 and impose no constraint. Hence there are

$$\underbrace{2N}_{\text{rows+columns}} + \underbrace{2(2N-3)}_{\text{diagonals, length}\ge 2} = 6N - 6 \text{ lines, of total length } 4N^2 - 4. \tag{2}$$

All three paradigms below model exactly these $6N - 6$ lines; they only differ in *how* a line constraint is expressed.

2.2. Propositional logic and SAT

A *literal* is a Boolean variable x or its negation $\neg x$. A *clause* is a disjunction (OR) of literals, and a formula in *conjunctive normal form* (CNF) is a conjunction (AND) of clauses. The SAT problem asks whether a CNF formula has an assignment that makes every clause true; a solver answers SAT (with a model) or UNSAT. Solvers read the DIMACS format, where each clause is a line of signed integers ending with 0. For example, "at most one of x_1, x_2, x_3" in pairwise form is

```
p cnf 3 3     -1 -2 0     -1 -3 0     -2 -3 0
```

Every formula has an equivalent CNF, but direct conversion can grow exponentially. Introducing *auxiliary variables* (as in the Tseitin transformation) keeps the CNF small. The resulting formula is *equisatisfiable*: it is satisfiable exactly when the original one is, and every model restricted to the original variables is a model of the original formula. All AMO encodings in Section 2.4 except pairwise use this idea.

2.3. How CDCL solvers work

DPLL and unit propagation. Classic DPLL search repeats three steps: *decide* (assign a value to a free variable), *unit propagation* (UP) and *backtrack* on conflict. A clause is *unit* when all its literals but one are false; the last literal must then be true. UP applies this rule until no unit clause is left or a clause becomes fully false (a *conflict*). UP typically takes more than 90% of the run time of a solver. This is why the shape of an encoding (how many clauses each assignment touches, and how long the chains of implications are) matters so much in practice.

Conflict-driven clause learning. CDCL [11] extends DPLL with the following components (Algorithm 1):

- **Decision levels and implication graph.** Every decision opens a new level. Each propagated literal stores the clause that implied it (its *reason*).
- **Conflict analysis (1UIP).** On a conflict the solver resolves the conflicting clause with reason clauses backwards until exactly one literal of the current level remains (the *first unique implication point*). The result is a new *learned clause* that is implied by the formula.
- **Non-chronological backjumping.** The solver jumps back to the second-highest level in the learned clause, where the clause becomes unit and immediately propagates.
- **Heuristics.** VSIDS-style branching prefers variables that appeared in recent conflicts; *phase saving* reuses a variable's last value; *restarts* restart the search while keeping learned clauses; *clause deletion* removes learned clauses of low quality, e.g. measured by literal block distance (LBD) [1].
- **Two watched literals** [12]: each clause is only visited when one of its two watched literals becomes false, which makes UP fast on large formulas.

Worked example: CDCL on 4-Queens. Fig. 2 follows CDCL on $N = 4$ with the pairwise encoding.

(a) The solver decides $x_{00} = 1$ at level 1. UP on the AMO clauses falsifies the rest of row 0, column 0 and the main diagonal. No ALO clause becomes unit yet.

(b) It decides $x_{12} = 1$ at level 2. UP falsifies x_{21} (anti-diagonal) and x_{23} (diagonal), while x_{20}, x_{22} were already false at level 1. The ALO clause of row 2, $(x_{20} \vee x_{21} \vee x_{22} \vee x_{23})$, is now false: a conflict. (Depending on propagation order the conflict may be detected on another clause first.)

(c) Resolving the conflicting clause with the reasons $(\neg x_{12} \vee \neg x_{21})$ and $(\neg x_{12} \vee \neg x_{23})$ yields the 1UIP clause $(x_{20} \vee x_{22} \vee \neg x_{12})$. The solver backjumps to level 1, where this clause is unit and forces $x_{12} = 0$. UP then forces $x_{13} = 1$ (row 1 has no other free cell) and $x_{21} = 1$ (row 2), after which row 3 has no free cell: a new conflict at level 1.

(d) Every path in the implication graph of this conflict starts at the decision x_{00}, so analysis learns the unit clause $(\neg x_{00})$: *no 4-Queens solution has a queen in a corner*. The solver continues at level 0 and soon finds a solution.

The example shows how a CDCL solver turns a failed branch into a reusable fact (a learned clause), instead of rediscovering the same conflict in other branches as plain backtracking would.

Algorithm 1 CDCL (simplified)

Require: CNF formula F
$level \leftarrow 0$
loop
  $conflict \leftarrow$ UNITPROPAGATE()
  if $conflict$ **then**
    if $level = 0$ **then return** UNSAT
    end if
    $(C, b) \leftarrow$ ANALYZECONFLICT() ▷ 1UIP learned clause C, backjump level b
    $F \leftarrow F \cup \{C\}$; BACKJUMP($b$); $level \leftarrow b$
  else if all variables are assigned **then**
    return SAT
  else
    occasionally RESTART() and REDUCELEARNEDCLAUSES()
    $\ell \leftarrow$ DECIDE() ▷ VSIDS + phase saving
    $level \leftarrow level + 1$; assign ℓ
  end if
end loop

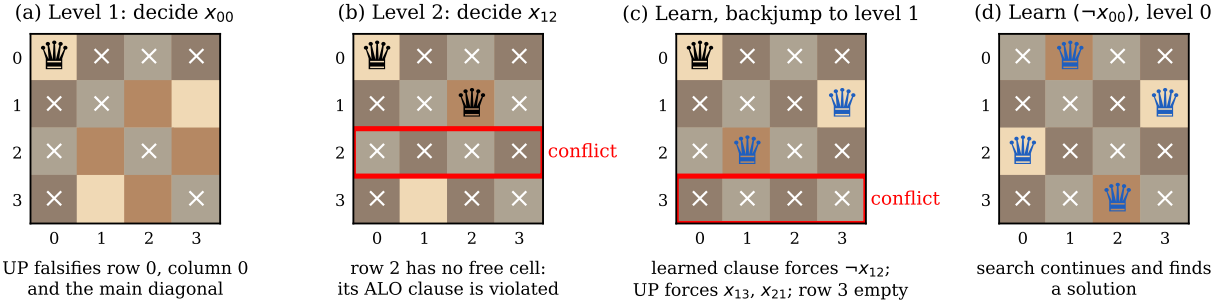


Figure 2: CDCL on 4-Queens (pairwise encoding). Black queens are decisions, blue queens are forced by unit propagation, grey cells are falsified by unit propagation, and red frames mark the clause that caused the conflict.

Solvers used. All solvers used here are CDCL solvers, but they differ in heuristics and in how much *inprocessing* (formula simplification during search) they perform (Table 1). We use them through PySAT [8], except CryptoMiniSat [22], which we call through its own Python binding.

2.4. AMO and ALO encodings

For Boolean variables $x_1, \dots, x_m$, the **at-least-one** (ALO) constraint is the single clause $x_1 \vee \cdots \vee x_m$, and "exactly one" = ALO + AMO. The **at-most-one** (AMO) constraint can be encoded in many ways. We implement five; the precise rules below are those of our implementation (they reproduce all counts in Table 8 exactly).

- **Pairwise (binomial).** For every pair $i < j$ add $(\neg x_i \vee \neg x_j)$. No auxiliary variables, $\binom{m}{2}$ clauses.
- **Binary** [7]. Add $k = \lceil \log_2 m \rceil$ variables $y_1, \dots, y_k$. Variable x_i ($i = 1..m$) receives the k-bit code of $i - 1$, and for each bit j we add $(\neg x_i \vee y_j)$ if the bit is 1 and $(\neg x_i \vee \neg y_j)$ if it is 0. Thus a true x_i "writes" its code into y. In total $m\lceil \log_2 m \rceil$ clauses.
- **Sequential counter** [21]. Add $m-1$ variables s_i meaning "some of $x_1, \dots, x_i$ is true": $(\neg x_1 \vee s_1)$; for $1 < i < m$: $(\neg x_i \vee s_i)$, $(\neg s_{i-1} \vee s_i)$, $(\neg s_{i-1} \vee \neg x_i)$; and $(\neg s_{m-1} \vee \neg x_m)$. In total $3m - 4$ clauses.
- **Commander** [9]. Split the variables into consecutive groups of $k = 3$. For each group G add a commander c with $c \leftrightarrow \bigvee G$, i.e. $(\neg c \vee \bigvee G)$ and $(\neg x \vee c)$ for $x \in G$, plus a pairwise AMO inside G. (A group of size 1 uses

Table 1
Main characteristics of the SAT solvers used.

Solver	Main characteristics
MiniSat 2.2 [6]	Compact reference CDCL solver: VSIDS, 1UIP learning with clause minimisation, Luby restarts, phase saving. Little extra machinery.
Glucose 4	MiniSat-based; ranks learned clauses by LBD, deletes low-quality clauses aggressively and restarts dynamically based on recent LBD values [1].
CaDiCaL 1.5.3 / 1.9.5 [3]	Modern solver with heavy inprocessing (vivification, subsumption, variable elimination, probing). Alternates between a "focused" mode (frequent restarts) and a "stable" mode (rare restarts).
Kissat 4.0.4 [3]	Re-implementation of CaDiCaL's ideas in C with compact data structures; winner of several recent SAT competitions.
Lingeling	Predecessor of CaDiCaL by the same author; also inprocessing-heavy.
MapleChrono	Maple family (learning-rate-based branching [17]) combined with chronological backtracking [18]; tuned for hard competition instances.
CryptoMiniSat 5 [22]	Detects XOR constraints and reasons about them with Gaussian elimination, plus inprocessing.

its variable as the commander.) Then apply the same construction recursively to the commanders until at most 4 remain, which are encoded pairwise.

- **Product** [5]. Place the variables row by row on a $p \times q$ grid ($p = \lceil\sqrt{m}\rceil$, $q = \lceil m/p \rceil$). Add row variables r_u and column variables c_v with $(\neg x \vee r_u)$ and $(\neg x \vee c_v)$ for the variable x in cell (u, v). Then apply AMO recursively to $r_1..r_p$ and to $c_1..c_q$; lists of at most 4 variables are encoded pairwise.

Example. Table 2 lists the full clause sets for $m = 4$. Pairwise forbids each pair directly. Binary forbids two true variables because they would write different codes into y. Sequential counter forbids them because the first true variable switches on the counter, and a switched-on counter forbids every later variable. Fig. 3 shows the structure of four encodings for larger m.

Table 2
Complete AMO clause sets for $m = 4$ variables $x_1..x_4$.

Encoding	Aux. variables	Clauses
Pairwise	–	$(\neg x_1 \vee \neg x_2)$, $(\neg x_1 \vee \neg x_3)$, $(\neg x_1 \vee \neg x_4)$, $(\neg x_2 \vee \neg x_3)$, $(\neg x_2 \vee \neg x_4)$, $(\neg x_3 \vee \neg x_4)$ (6)
Binary	y_1, y_2	x_1: 00: $(\neg x_1 \vee \neg y_1)$, $(\neg x_1 \vee \neg y_2)$; x_2: 01: $(\neg x_2 \vee y_1)$, $(\neg x_2 \vee \neg y_2)$; x_3: 10: $(\neg x_3 \vee \neg y_1)$, $(\neg x_3 \vee y_2)$; x_4: 11: $(\neg x_4 \vee y_1)$, $(\neg x_4 \vee y_2)$ (8)
Sequential	s_1, s_2, s_3	$(\neg x_1 \vee s_1)$; $(\neg x_2 \vee s_2)$, $(\neg s_1 \vee s_2)$, $(\neg s_1 \vee \neg x_2)$; $(\neg x_3 \vee s_3)$, $(\neg s_2 \vee s_3)$, $(\neg s_2 \vee \neg x_3)$; $(\neg s_3 \vee \neg x_4)$ (8)

Correctness.

Proposition 1. *For each of the five encodings, an assignment of $x_1..x_m$ can be extended to the auxiliary variables so that all clauses are satisfied* if and only if *at most one x_i is true.*

Proof sketch. ($\Leftarrow$) If at most one x_a is true, set the auxiliary variables "honestly": y = code of a (binary); $s_i = x_1 \vee \cdots \vee x_i$ (sequential); $c = \bigvee G$ (commander); r_u, c_v true only for the cell of x_a (product). Every clause is then satisfied. ($\Rightarrow$) Suppose x_a and x_b ($a < b$) are both true. *Pairwise:* clause $(\neg x_a \vee \neg x_b)$ is violated. *Binary:* the codes of a and b differ in some bit j, so y_j would have to be both true and false. *Sequential:* $x_a \Rightarrow s_a \Rightarrow s_{a+1} \Rightarrow \cdots \Rightarrow s_{b-1} \Rightarrow \neg x_b$. *Commander:* in the same group the pairwise clauses are violated; in different groups two commanders are true, which the recursive AMO forbids. *Product:* two different cells differ in their row or column, so two row variables or two column variables are true, which the recursive AMO forbids. □

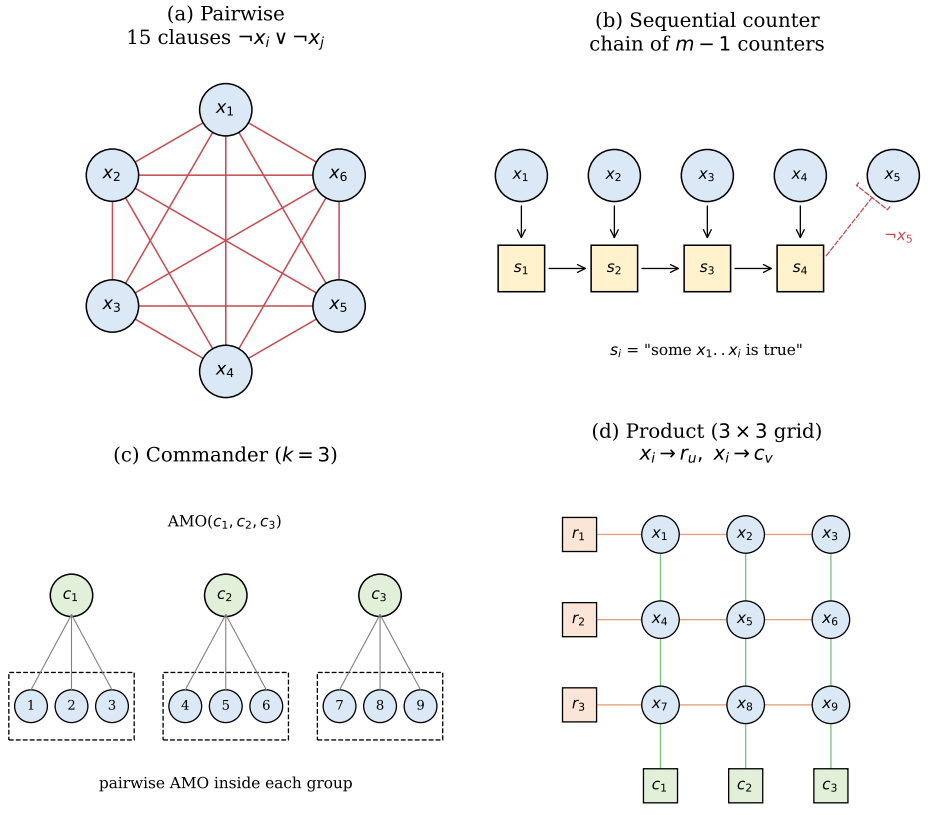


Figure 3: Structure of the AMO encodings. (a) Pairwise: one clause per pair (red edges). (b) Sequential counter: x_i sets counter s_i, counters propagate to the right, and a set counter forbids later variables. (c) Commander: one commander per group of 3, AMO applied recursively to the commanders. (d) Product: each x_i implies its row variable r_u and column variable c_v; AMO is then applied to the rows and to the columns.

We additionally checked Proposition 1 by brute force over all assignments for $m \leq 9$; the experimental check on the full N-Queens formulas is given in Section 5.1.

Propagation strength. For a CDCL solver it is not enough that an encoding is correct; it should also let *unit propagation* draw the obvious conclusions.

Proposition 2. *In all five encodings, setting $x_a = 1$ lets unit propagation alone derive $x_b = 0$ for every $b \neq a$.*

Proof sketch. Pairwise: directly. Binary: x_a fixes all y_j; for every $b \neq a$ some clause $(\neg x_b \vee \pm y_j)$ now has a false second literal. Sequential: forwards, $x_a \Rightarrow s_a \Rightarrow \cdots \Rightarrow s_{m-1}$ and $s_{b-1} \Rightarrow \neg x_b$ for $b > a$; backwards, $(\neg s_{a-1} \vee \neg x_a)$ gives $\neg s_{a-1}$, then $(\neg s_{i-1} \vee s_i)$ gives $\neg s_{a-2}, \ldots, \neg s_1$, and $(\neg x_b \vee s_b)$ gives $\neg x_b$ for $b < a$. Commander and product: by induction over the recursion levels. □

So no encoding is "weaker" than another in this sense. The encodings do differ in *how many propagation steps* are needed. Table 3 reports the number of UP rounds needed to derive all $x_b = 0$ after $x_a = 1$ (worst case over a), which we call the *propagation depth*. It is constant for pairwise and binary, grows logarithmically for commander and product, but grows *linearly* for the sequential counter, whose information must travel along a chain of counter variables. We use this property in Section 6 to explain the measured run times.

PySAT built-in encodings. PySAT's `CardEnc.atmost(lits, bound=1, encoding=...)` offers several AMO encodings, of which we use four: `pairwise`; `seqcounter` (Sinz's sequential counter [21]); `bitwise` (the binary encoding [7]); and `ladder` (the ladder encoding of Gent and Nightingale [14], a chain of $m - 1$ auxiliary variables closely related to the

Table 3
Size and propagation depth of each AMO encoding over m variables. The last columns give exact values for a line of $m = 64$ variables (one row of the 64×64 board).

				$m = 64$		
Encoding	Aux. variables	Clauses	Prop. depth	Aux.	Clauses	Depth
Pairwise	0	$m(m-1)/2$	1	0	2016	1
Binary	$\lceil \log_2 m \rceil$	$m\lceil \log_2 m \rceil$	2	6	384	2
Sequential counter	$m-1$	$3m-4$	$\Theta(m)$	63	188	64
Commander ($k = 3$)	$\approx m/2$	$\approx 3.5m$	$O(\log m)$	31	217	7
Product	$\approx 2\sqrt{m}$	$\approx 2m + O(\sqrt{m})$	$O(\log \log m)$	28	172	5

sequential counter). They serve as an independent reference for our own implementations (Appendix **??** shows how they are called).

2.5. Constraint programming

Let the integer variable $q_i \in \{0, \dots, N-1\}$ be the column of the queen in row i. Then "one queen per row" holds by construction, and the remaining constraints are [13]:

$$\texttt{AllDifferent}(q_0, \dots, q_{N-1}), \quad \texttt{AllDifferent}(q_0+0, \dots, q_{N-1}+N-1), \quad \texttt{AllDifferent}(q_0-0, \dots, q_{N-1}-(N-1)). \tag{3}$$

The first constraint covers columns, the second anti-diagonals and the third diagonals. Only N variables are needed, compared with N^2 for SAT and ILP.

Why a global constraint helps. `AllDifferent` could be written as $\binom{N}{2}$ pairwise disequalities $q_i \neq q_j$, but these see only two variables at a time. Consider $q_0, q_1 \in \{1, 2\}$ and $q_2 \in \{1, 2, 3\}$. No single disequality removes a value. The global constraint, however, notices that q_0 and q_1 use up the two values $\{1, 2\}$ (a *Hall set* [15]), so $q_2 = 3$. Régin's algorithm [13] finds all such deductions (generalised arc consistency) in polynomial time by computing a maximum matching between variables and values. This is the CP counterpart of choosing a strong AMO encoding in SAT: the whole line is reasoned about at once.

The two CP solvers. IBM CP Optimizer [16] combines constraint propagation with tree search and restarts; its `AllDifferent` filtering strength is configurable. *Google OR-Tools CP-SAT* [20] is a *lazy clause generation* solver [19]: integer domains are represented by Boolean literals such as $[q_i = v]$ or $[q_i \leq v]$, created on demand; every propagator explains its deductions as clauses, so that CDCL-style learning (Section 2.3) also works on the CP model. In multi-threaded mode CP-SAT runs a portfolio of different strategies; here it runs with one worker. We test two CP-SAT models: the integer model of Eq. (3) ("CP-SAT int") and a Boolean model with N^2 variables, `AddExactlyOne` for rows/columns and `AddAtMostOne` for diagonals ("CP-SAT bool"), which mirrors the SAT model.

2.6. Integer linear programming

Let $x_{ij} \in \{0, 1\}$ be 1 iff a queen is on (i, j). The feasibility model is:

$$\sum_j x_{ij} = 1 \qquad \forall i \quad \text{(rows)} \tag{4}$$

$$\sum_i x_{ij} = 1 \qquad \forall j \quad \text{(columns)} \tag{5}$$

$$\sum_{(i,j)\in D} x_{ij} \leq 1 \qquad \forall \text{ diagonal } D \text{ with } |D| \geq 2. \tag{6}$$

There is no objective: the solver stops at the first integer-feasible solution. The model has N^2 binary variables and $6N - 6$ constraints (Eq. (2)).

How branch-and-bound/cut solves it. The solver first solves the *LP relaxation*, in which $0 \leq x_{ij} \leq 1$ may be fractional. If the LP solution is integral, it is a solution. Otherwise the solver strengthens the relaxation with *cutting planes*, tries *primal heuristics* (rounding, diving, local search) to find an integral point, and *branches* on a fractional variable ($x_{ij} = 0$ vs. $x_{ij} = 1$), solving the two sub-problems recursively [23].

The LP relaxation is uninformative. Setting every $x_{ij} = 1/N$ satisfies all constraints: each row and column sums to 1, and a diagonal of length $L \le N$ sums to $L/N \le 1$. So the relaxation is always feasible and can return a "uniform" fractional point that says nothing about where queens should go. For N-Queens the work of an ILP solver therefore lies almost entirely in presolve, heuristics and branching, not in the LP bound. Note also that each diagonal inequality (6) is already a *clique inequality*, the strongest linear form of "at most one". It plays the same role as a compact AMO encoding: writing the same constraint as pairwise inequalities $x_a + x_b \le 1$ would give a weaker relaxation and $\Theta(N^3)$ rows, just as pairwise does in SAT.

3. Methodology

3.1. SAT model

Let x_{ij} (DIMACS index $iN + j + 1$) be true iff a queen is on (i, j). The CNF is:

$$\bigvee_j x_{ij} \ \wedge\ \mathrm{AMO}(x_{i0}, \dots, x_{i,N-1}) \qquad \forall i \quad \text{(rows)} \tag{7}$$

$$\bigvee_i x_{ij} \ \wedge\ \mathrm{AMO}(x_{0j}, \dots, x_{N-1,j}) \qquad \forall j \quad \text{(columns)} \tag{8}$$

$$\mathrm{AMO}(\{x_{ij} : i - j = d\}),\ \mathrm{AMO}(\{x_{ij} : i + j = s\}) \qquad \text{(diagonals of length} \ge 2) \tag{9}$$

This gives N^2 main variables, $2N$ ALO clauses and $6N - 6$ AMO constraints. Replacing "AMO" by any encoding of Section 2.4 gives one SAT variant. Auxiliary variables are numbered from $N^2 + 1$.

Closed-form CNF sizes. Using the line lengths of Eq. (2) (rows and columns: $2N$ lines of length N; each diagonal direction: one line of length N and two of each length $2, \dots, N - 1$):

- *Pairwise.* $\sum_L \binom{|L|}{2} = 2N\binom{N}{2} + 2\left[\binom{N}{2} + 2\sum_{\ell=2}^{N-1}\binom{\ell}{2}\right] = N^2(N-1) + N(N-1) + 4\binom{N}{3} = \dfrac{N(N-1)(5N-1)}{3}$ AMO clauses, i.e. $\Theta(N^3)$.
- *Sequential counter.* $\sum_L (3|L| - 4) = 3(4N^2 - 4) - 4(6N - 6) = 12(N-1)^2$ AMO clauses and $\sum_L (|L| - 1) = 2(2N - 1)(N - 1)$ auxiliary variables, i.e. $\Theta(N^2)$.
- *Binary, commander, product* likewise give $\Theta(N^2 \log N)$, $\Theta(N^2)$ and $\Theta(N^2)$ clauses, respectively, by summing Table 3 over all lines.

The formulas match the implementation exactly (e.g. pairwise, $N = 64$: 428,736 + 128 = 428,864 clauses; sequential, $N = 8$: 588 + 16 = 604 clauses).

3.2. Model size per paradigm

Table 4 compares the three models. It also explains the licence limits observed later. Gurobi's community licence allows 2000 variables and CPLEX's 1000, so the N^2-variable ILP model fits only up to $N = 44$ ($44^2 = 1936$) and $N = 31$ ($31^2 = 961$). CP Optimizer's Community Edition limits the search space to about 2^{1000}. If the search space is measured as the product of domain sizes, $N^N \le 2^{1000}$, i.e. $N \log_2 N \le 1000$, which holds up to $N = 140$. This is consistent with our observation that $N = 128$ runs ($128 \log_2 128 = 896$) while $N = 200$ is refused (≈ 1529).

Table 4
Model size of each paradigm for an $N \times N$ board.

Model	Variables	Constraints	Largest N under commu
SAT, pairwise	N^2 Boolean	$\Theta(N^3)$ clauses	no limit (open source)
SAT, other encodings	$N^2 + \Theta(N^2)$ aux.	$\Theta(N^2)$ or $\Theta(N^2 \log N)$ clauses	no limit (open source)
CP int (CP-SAT, CPLEX CP)	N integer, domain size N	3 `AllDifferent`	CP-SAT: none; CPLEX
CP-SAT bool	N^2 Boolean	$2N$ ExactlyOne + $(4N - 6)$ AtMostOne	no limit
ILP (Gurobi, CPLEX MIP)	N^2 binary	$6N - 6$ linear	Gurobi: 44; CPLEX: 31

3.3. Solving procedure

Algorithm 2 shows the SAT pipeline. Clauses are streamed directly into the solver's C/C++ clause database instead of first being stored as a Python list, which greatly reduces peak memory. Every returned solution, from any solver, is checked by the same independent validator.

Algorithm 2 N-Queens via SAT

Require: board size N, AMO encoding *enc*, SAT solver S
Ensure: a list of N cells, or UNSAT

$S \leftarrow$ new solver; $top \leftarrow N^2$
for each line L (row, column, diagonal, anti-diagonal) with $|L| \geq 2$ **do**
  $lits \leftarrow \{x_{ij} : (i, j) \in L\}$
  if L is a row or a column **then**
    add ALO clause $\bigvee lits$
  end if
  add the clauses of $\text{AMO}_{enc}(lits)$ to S; update top
end for
if S.solve() = SAT **then**
  $Q \leftarrow \{(i, j) : x_{ij}$ is true$\}$; check rows, columns, $i - j$, $i + j$ are all distinct
  return Q
else
  return UNSAT
end if

3.4. Compared methods

Fig. 4 gives an overview of the experimental pipeline.

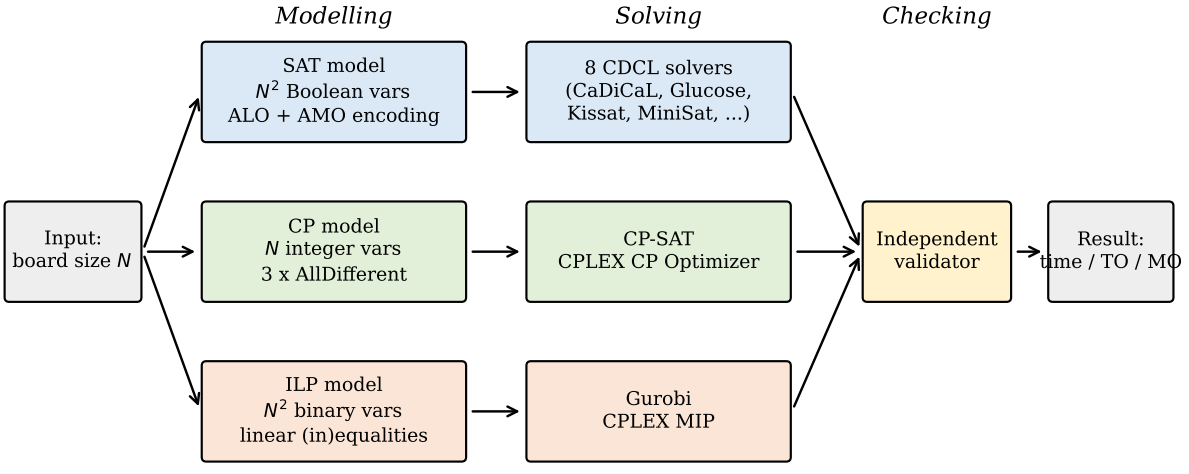


Figure 4: Overview of the study: the same instance is modelled in three paradigms, solved by the corresponding solvers, and every solution is checked by one independent validator.

Table 5 lists all methods.

Table 5
Methods compared in this study.

Paradigm	Solver / tool	Model	Variables
SAT	8 CDCL solvers (PySAT, pycryptosat)	CNF, Eqs. (7)–(9)	N^2 Boolean + auxiliary
CP	IBM CP Optimizer (CPLEX CP)	`AllDifferent`, Eq. (3)	N integer
CP	OR-Tools CP-SAT (int)	`AllDifferent`, Eq. (3)	N integer
CP	OR-Tools CP-SAT (bool)	ExactlyOne / AtMostOne	N^2 Boolean
ILP	Gurobi	Eqs. (4)–(6)	N^2 binary
ILP	CPLEX MIP	Eqs. (4)–(6)	N^2 binary

4. Experimental setup

Hardware and software. Ubuntu 24.04 virtual machine, 1 vCPU, 3 GB RAM, Python 3.12. Software: python-sat 1.9.dev15 (CaDiCaL 1.5.3 and 1.9.5, Glucose 4, MiniSat 2.2, MapleChrono, Lingeling, Kissat 4.0.4), pycryptosat 5.16, OR-Tools 9.15 (CP-SAT), Gurobi 13.0.3, CPLEX 22.2 and CP Optimizer 22.2.

Protocol.

- Each run is a fresh OS process with a **30-second** wall-clock limit (model building + solving) and a 2.7 GB memory limit.
- TO = time-out (over 30 s); MO = memory-out; N/A = model refused by the licence.
- All solvers use **one thread**; runs are executed strictly one after another.
- Each configuration is run once. Small time differences between neighbouring cells should therefore not be over-interpreted.
- Metric: total time in seconds (encoding + loading + solving).

Licence limits. The commercial solvers are used with free community/evaluation licences, which limit the model size (Table 6). A N/A entry is a licence limit, not a solver failure.

Table 6
Model-size limits of the community/evaluation licences.

Solver	Licence limit	Largest N allowed
Gurobi	≤ 2000 variables	$N \leq 44$
CPLEX MIP	≤ 1000 variables	$N \leq 31$
CP Optimizer	search space $\leq 2^{1000}$	$N \leq 128$ (fails at $N = 200$)

5. Results

5.1. Correctness

For $N = 2, \ldots, 9$ we enumerate all solutions (solve, block the found solution, repeat) and compare the count with the known values. All nine encodings give exactly the right counts (Table 7), and every solution passes the validator.

5.2. RQ1a – CNF size

Table 8 and Fig. 5 show the size of the CNF. At $N = 1024$, pairwise would need about 1.79 billion clauses (computed by the formula, not built), binary 41.2 million, while sequential, commander and product need only 9–15 million. Product has the fewest clauses at every size, followed by sequential; however, sequential uses the most auxiliary variables (5.2 million at $N = 1024$, vs. 3.1 million for commander and 1.6 million for product).

Table 7
Number of solutions found by exhaustive enumeration (CaDiCaL 1.5.3).

N	2	3	4	5	6	7	8	9
Known value (OEIS A000170)	0	0	2	10	4	40	92	352
Our 5 encodings (pairwise, binary, sequential, commander, product)	0	0	2	10	4	40	92	352
PySAT encodings (pairwise, seqcounter, bitwise, ladder)	0	0	2	10	4	40	92	352

Table 8
Number of variables and clauses per encoding. †: computed from the closed-form formula, not built.

	$N = 64$		$N = 256$		$N = 1024$	
Encoding	#vars	#clauses	#vars	#clauses	#vars	#clauses
Pairwise	4,096	428,864	65,536	27,831,552†	1,048,576	1,787,474,944†
Binary	6,136	95,576	76,788	2,053,464	1,105,904	41,243,992
Sequential	20,098	47,756	326,146	780,812	5,236,738	12,560,396
Commander	11,906	54,918	195,214	907,754	3,142,708	14,645,478
Product	12,476	**45,620**	125,800	**631,720**	1,578,860	**9,324,048**

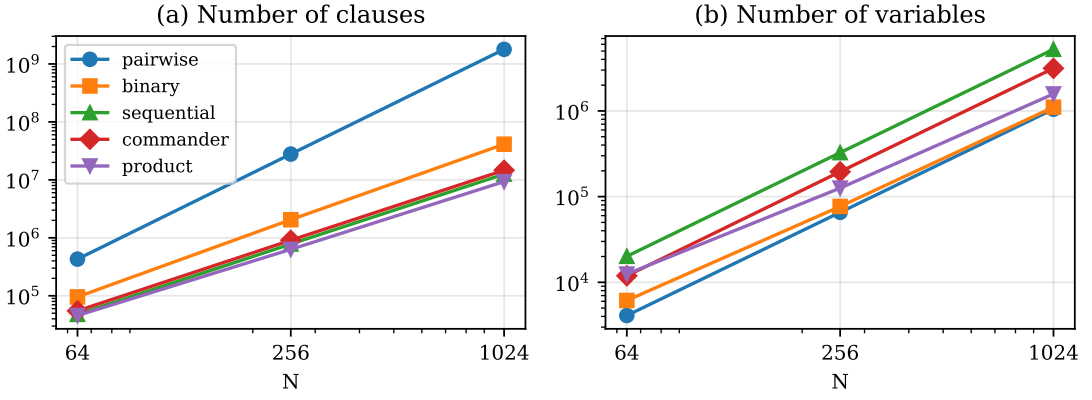


Figure 5: CNF size vs. N (log–log scale).

5.3. RQ1b – Solving time per encoding

Table 9 and Fig. 6 give the total time with CaDiCaL 1.5.3.

- **Small boards ($N \leq 32$):** every encoding finishes in under 1 s.
- **Commander** is the fastest or nearly fastest from $N = 64$ on (0.20 s at $N = 64$, 11.4 s at $N = 512$). Together with binary, it is the only encoding that solves both $N = 400$ and $N = 512$.
- **Binary** also reaches $N = 512$ (24.1 s) but runs out of memory at $N = 800$.
- **Pairwise** is competitive up to $N = 128$ but runs out of memory from $N = 256$, as expected from its $\Theta(N^3)$ clauses.
- **Sequential counter** has the second-fewest clauses but is the slowest from $N = 64$ (5.75 s vs. 0.20 s for commander) and times out from $N = 256$.
- **Product** is not monotonic: it solves $N = 512$ (23.9 s) but times out at $N = 400$, which illustrates the run-to-run variance of CDCL search.

Table 9
Total time (s) per AMO encoding, CaDiCaL 1.5.3. Best time per row in bold.

N	Pairwise	Binary	Sequential	Commander	Product
8	0.10	**0.08**	0.13	0.12	0.09
16	0.16	0.15	0.25	0.15	**0.14**
32	0.95	0.47	0.44	**0.20**	0.44
64	2.09	3.93	5.75	**0.20**	0.94
100	5.46	2.18	4.22	**0.58**	2.91
128	7.91	1.19	8.82	**0.41**	1.38
200	17.40	3.82	19.09	2.66	**1.97**
256	MO	5.82	TO	**1.87**	3.54
400	MO	13.27	TO	**7.47**	TO
512	MO	24.06	TO	**11.36**	23.90
800	MO	MO	TO	TO	TO
1024	MO	MO	MO	TO	TO

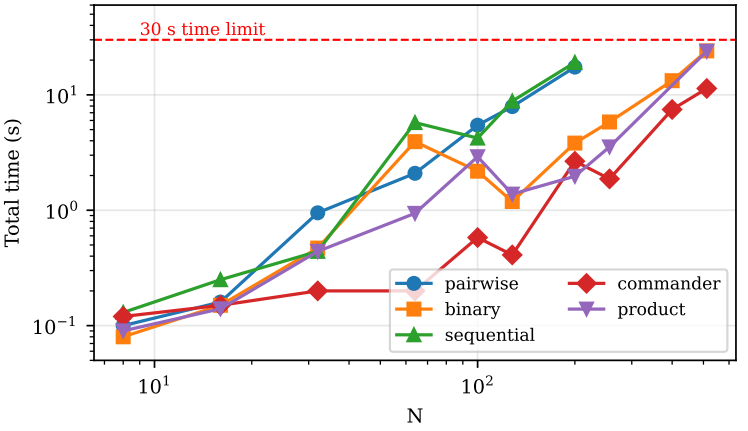


Figure 6: Total time vs. N per encoding (CaDiCaL 1.5.3; failed runs omitted).

Fig. 7 presents the same data as a status map, which makes the "scalability frontier" of each encoding easy to see: commander and binary stay solvable (coloured) the longest, while pairwise turns into memory-outs and sequential into time-outs already at $N = 256$.

PySAT built-in encodings. Table 10 shows PySAT's own encodings. They behave similarly to our implementations of the same families, which shows that our results are not caused by slow Python code. Bitwise (PySAT's binary-style encoding) is the most robust and solves $N = 512$ in 22.3 s.

Table 10
Total time (s) for PySAT's built-in AMO encodings, CaDiCaL 1.5.3.

N	pairwise	seqcounter	bitwise	ladder
64	**0.47**	1.08	0.56	0.77
128	5.07	2.33	**2.44**	2.76
256	MO	TO	**6.05**	10.43
512	MO	TO	**22.28**	TO

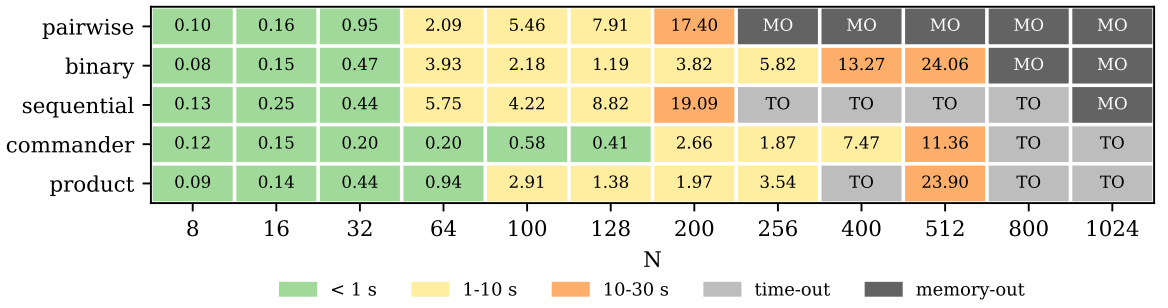


Figure 7: Status map of Table 9: each cell shows the total time (s), coloured by speed; grey cells are time-outs (TO) or memory-outs (MO).

Key finding

RQ1. The commander encoding is the most robust overall. Fewer clauses do *not* mean faster solving: sequential counter is very compact but among the slowest, while pairwise fails because of memory ($\Theta(N^3)$ clauses).

5.4. RQ2 – Comparing SAT solvers

Table 11 compares eight solvers on the two most robust encodings for large N (commander and product). A "–" means the solver had already timed out at a smaller N and was not run again.

- **Commander:** Glucose4 is the only solver that solves $N = 1024$ (29.5 s). MiniSat 2.2 is the fastest at $N = 256$ and $N = 512$ (1.08 s and 6.71 s). Lingeling, both CaDiCaL versions and Kissat solve $N = 512$ in 9–15 s.
- **Product:** MiniSat is the only solver that solves $N = 1024$ (29.9 s); Lingeling (3.5 s) and MiniSat (4.0 s) are the fastest at $N = 512$.
- CryptoMiniSat is clearly slower (7.2 s vs. 1.1 s for MiniSat at $N = 256$, commander) and times out from $N = 512$.
- MapleChrono, which is tuned for hard (often UNSAT) instances, already times out at $N = 128$.

Table 11
Total time (s) per SAT solver and encoding. "–": not run because the solver already timed out at a smaller N.

	Commander				Product			
Solver	128	256	512	1024	128	256	512	1024
CaDiCaL 1.5.3	1.16	1.61	10.98	TO	1.48	2.79	22.50	TO
CaDiCaL 1.9.5	0.37	1.91	12.58	TO	2.12	3.66	TO	–
Glucose 4	0.39	1.57	10.39	**29.52**	TO	–	–	–
MiniSat 2.2	0.29	**1.08**	**6.71**	TO	**0.20**	**0.73**	3.96	**29.93**
MapleChrono	TO	–	–	–	TO	–	–	–
Lingeling	0.59	2.40	9.45	TO	0.61	0.93	**3.52**	TO
Kissat 4.0.4	0.44	2.42	14.82	TO	1.58	3.07	5.42	TO
CryptoMiniSat 5	**0.26**	7.22	TO	–	0.45	17.97	TO	–

Key finding

RQ2. Simple, lightly tuned solvers (MiniSat, Glucose) perform best on this easy, highly symmetric satisfiable problem. This ranking is specific to N-Queens and should not be read as a general solver ranking.

5.5. RQ3 – SAT vs. CP vs. ILP

Table 12 and Fig. 8 compare the best SAT configuration (commander + MiniSat) with the CP and ILP solvers. Model-building time is included.

Table 12
Total time (s) of SAT, CP and ILP solvers (30 s limit, one thread). Best per row in bold. *CPLEX MIP at $N \geq 1000$ ran out of time while still building the N^2-variable model.

	SAT	CP			ILP	
N	commander+MiniSat	CP-SAT (int)	CP-SAT (bool)	CPLEX CP	Gurobi	CPLEX MIP
8	0.03	0.18	**0.01**	1.31	0.33	0.43
16	0.02	0.02	0.01	0.01	**<0.01**	0.01
24	0.03	0.03	0.03	**0.01**	0.04	**0.01**
31	0.06	0.06	0.05	**0.01**	**0.01**	**0.01**
32	0.04	0.06	0.05	**0.01**	0.02	N/A
44	0.05	0.12	0.10	0.02	**0.01**	N/A
64	0.10	0.30	0.28	**0.01**	N/A	N/A
100	0.18	1.03	0.81	**0.01**	N/A	N/A
128	0.31	1.97	1.78	**0.02**	N/A	N/A
200	**0.72**	16.11	2.47	N/A	N/A	N/A
256	**1.10**	TO	3.54	N/A	N/A	N/A
512	**6.60**	TO	TO	N/A	N/A	N/A
1000	**27.69**	TO	TO	N/A	N/A	TO*
2000	MO	TO	MO	N/A	N/A	TO*

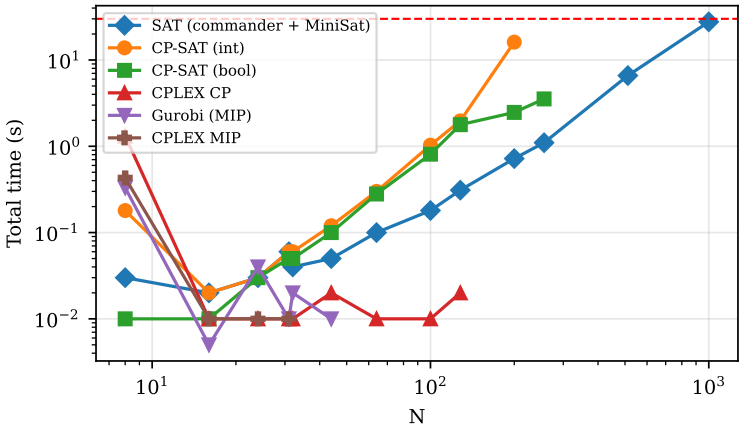


Figure 8: Total time vs. N for SAT, CP and ILP (successful runs only; "<0.01" plotted as 0.005).

- **CPLEX CP is the fastest where it can run.** For $16 \leq N \leq 128$ it needs only 0.01–0.02 s, about 10–20 times faster than SAT (e.g. 0.02 s vs. 0.31 s at $N = 128$). The `AllDifferent` propagator prunes far more per node than unit propagation on a flat CNF. (At $N = 8$ its 1.31 s is most likely dominated by start-up overhead.)
- **Gurobi and CPLEX MIP** are also fast (mostly 0.01–0.04 s) up to their licence limits ($N = 44$ and $N = 31$). We cannot draw conclusions beyond these limits.
- **SAT scales further than CP-SAT.** CP-SAT (int) needs 16.1 s at $N = 200$ and times out from $N = 256$; CP-SAT (bool) reaches $N = 256$ (3.5 s) but times out at $N = 512$. SAT solves $N = 512$ in 6.6 s and $N = 1000$ in 27.7 s. Note that CP-SAT was run single-threaded; its default multi-threaded mode may behave very differently.

- **No exact method solves** $N = 2000$ within the limits: SAT and CP-SAT (bool) run out of memory.

Key finding

RQ3. Within licence limits, CPLEX CP is the fastest exact method. Among methods that can run at every size, SAT (commander + MiniSat) scales furthest, reaching $N = 1000$, about four times larger than single-threaded CP-SAT.

6. Discussion

Why clause count is not enough. Sequential counter and product produce the smallest CNFs, yet sequential is much slower than commander, and product is unstable across N. CDCL performance depends on how well unit propagation and clause learning work on the encoding's structure, not only on its size. Commander keeps clauses short and adds relatively few auxiliary variables, which seems to suit CDCL solvers well on this problem.

Why CP is fast. CP keeps the problem as three `AllDifferent` constraints over only N integer variables, while SAT and ILP need N^2 variables. The global propagator reasons about a whole line at once, so much less search is needed.

Threats to validity.

- *Licences:* the SAT vs. Gurobi/CPLEX comparison is only valid for $N \leq 44$ and $N \leq 31$. With full licences these solvers may scale very differently.
- *Single run, single core:* CDCL search has high variance between configurations, so small time differences are not statistically meaningful.
- *Single thread:* CP-SAT and the commercial solvers are designed to use several threads; our setting is fair but not their best case.
- *Scope:* we did not use symmetry-breaking clauses or more advanced cardinality encoders (e.g. PBLib).

7. Conclusion

We solved N-Queens with SAT using five hand-written and four PySAT AMO encodings on eight CDCL solvers, and compared SAT with CP-SAT, CPLEX CP, Gurobi and CPLEX MIP. All encodings were verified by exhaustive solution counting. Our main conclusions are:

1. **Encoding (RQ1):** commander is the most robust AMO encoding; small CNF size does not guarantee fast solving.
2. **Solvers (RQ2):** MiniSat and Glucose are the most robust solvers on this problem; they are the only ones to reach $N = 1024$.
3. **Paradigms (RQ3):** CPLEX CP is the fastest exact method within its licence ($N \leq 128$); SAT scales further than single-threaded CP-SAT and reaches $N = 1000$.

Future work: full commercial licences to extend the CP/ILP comparison to large N; symmetry-breaking SAT encodings and PBLib encoders; multi-threaded solvers; and repeated runs with several random seeds to measure variance.

Data Availability

Source code and experimental results are available at: `https://github.com/<your-username>/nqueens-solvers`.

References

[1] G. Audemard, L. Simon, Predicting learnt clauses quality in modern SAT solvers, in: Proc. IJCAI 2009, pp. 399–404.
[2] J. Bell, B. Stevens, A survey of known results and research areas for n-queens, Discrete Mathematics 309 (1) (2009) 1–31.
[3] A. Biere, K. Fazekas, M. Fleury, M. Heisinger, CaDiCaL, Kissat, Paracooba, Plingeling and Treengeling entering the SAT Competition 2020, in: Proc. SAT Competition 2020: Solver and Benchmark Descriptions, pp. 51–53.

[4] A. Biere, M. Heule, H. van Maaren, T. Walsh (Eds.), Handbook of Satisfiability, IOS Press, 2009.
[5] J. Chen, A new SAT encoding of the at-most-one constraint, in: Proc. 9th Int. Workshop on Constraint Modelling and Reformulation (ModRef), 2010.
[6] N. Eén, N. Sörensson, An extensible SAT-solver, in: SAT 2003, LNCS 2919, pp. 502–518.
[7] A. M. Frisch, P. A. Giannaros, SAT encodings of the at-most-k constraint: some old, some new, some fast, some slow, in: Proc. 9th Int. Workshop on Constraint Modelling and Reformulation (ModRef), 2010.
[8] A. Ignatiev, A. Morgado, J. Marques-Silva, PySAT: a Python toolkit for prototyping with SAT oracles, in: SAT 2018, LNCS 10929, pp. 428–437.
[9] W. Klieber, G. Kwon, Efficient CNF encoding for selecting 1 from N objects, in: Workshop on Constraints in Formal Verification, 2007.
[10] C. Letavec, J. Ruggiero, The n-queens problem, INFORMS Transactions on Education 2 (3) (2002) 101–103.
[11] J. P. Marques-Silva, I. Lynce, S. Malik, Conflict-driven clause learning SAT solvers, in: Handbook of Satisfiability, IOS Press, 2009.
[12] M. W. Moskewicz, C. F. Madigan, Y. Zhao, L. Zhang, S. Malik, Chaff: engineering an efficient SAT solver, in: Proc. DAC 2001, pp. 530–535.
[13] J.-C. Régin, A filtering algorithm for constraints of difference in CSPs, in: Proc. AAAI 1994, pp. 362–367.
[14] I. P. Gent, P. Nightingale, A new encoding of AllDifferent into SAT, in: Proc. 3rd Int. Workshop on Constraint Modelling and Reformulation (ModRef), 2004.
[15] P. Hall, On representatives of subsets, J. London Math. Soc. 1 (1) (1935) 26–30.
[16] P. Laborie, J. Rogerie, P. Shaw, P. Vilím, IBM ILOG CP Optimizer for scheduling, Constraints 23 (2) (2018) 210–250.
[17] J. H. Liang, V. Ganesh, P. Poupart, K. Czarnecki, Learning rate based branching heuristic for SAT solvers, in: Proc. SAT 2016, LNCS 9710, pp. 123–140.
[18] A. Nadel, V. Ryvchin, Chronological backtracking in SAT, in: Proc. SAT 2018, LNCS 10929, pp. 111–121.
[19] O. Ohrimenko, P. J. Stuckey, M. Codish, Propagation via lazy clause generation, Constraints 14 (3) (2009) 357–391.
[20] L. Perron, V. Furnon, OR-Tools, v9.15, Google, 2024.
[21] C. Sinz, Towards an optimal CNF encoding of Boolean cardinality constraints, in: CP 2005, LNCS 3709, pp. 827–831.
[22] M. Soos, K. Nohl, C. Castelluccia, Extending SAT solvers to cryptographic problems, in: SAT 2009, LNCS 5584, pp. 244–257.
[23] L. A. Wolsey, Integer Programming, John Wiley & Sons, 1998.